

# Spezifikation des Tools KlausurCraft: Werkzeug zur Erstellung von Klausuren

Version: 1.0

Verfasser: Simon Riedinger

Erstellt am: 06.10.2025

Letzte Änderung am:

## 1 [Inhaltsverzeichnis](#)

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>2</b> | <b>Beschreibung .....</b>                         | <b>3</b> |
| 2.1      | Zweck des Dokuments .....                         | 3        |
| 2.2      | Überblick über das System .....                   | 3        |
| 2.3      | Systemumgebung .....                              | 3        |
| 2.3.1    | Zielformat .....                                  | 3        |
| 2.3.2    | Hardwareanforderungen .....                       | 4        |
| 2.3.3    | Softwarevoraussetzungen .....                     | 4        |
| 2.3.4    | Installation .....                                | 4        |
| 2.4      | Nicht-funktionale Anforderungen .....             | 4        |
| 2.4.1    | Leistungsanforderungen .....                      | 4        |
| 2.4.2    | Entwurfseinschränkungen .....                     | 4        |
| 2.4.3    | Verfügbarkeit .....                               | 4        |
| 2.4.4    | Sicherheit .....                                  | 4        |
| 2.4.5    | Robustheit .....                                  | 5        |
| 2.4.6    | Wartbarkeit .....                                 | 5        |
| 2.4.7    | Portabilität .....                                | 5        |
| 2.5      | Beschreibung der Benutzer .....                   | 5        |
| 2.6      | Standardwerte .....                               | 5        |
| 2.6.1    | Allgemeine Standardwerte .....                    | 5        |
| 2.6.2    | Standardwerte für Tasks .....                     | 6        |
| 2.6.3    | Standardwerte für Subtasks .....                  | 6        |
| 2.6.4    | Standardwerte für Varianten .....                 | 6        |
| 2.6.5    | Standardwerte für generierte Exams .....          | 6        |
| <b>3</b> | <b>Spezifikation der Benutzeroberfläche .....</b> | <b>7</b> |
| 3.1      | Übersicht .....                                   | 7        |
| 3.2      | Hauptoberfläche .....                             | 8        |
| 3.2.1    | Navigationsleiste .....                           | 9        |
| 3.2.2    | Hauptbereich .....                                | 10       |
| 3.2.3    | Fußleiste .....                                   | 11       |
| 3.3      | Subtask .....                                     | 11       |
| 3.3.1    | Delete Subtask .....                              | 13       |
| 3.3.2    | Delete Variant .....                              | 13       |
| 3.4      | Edit Task Title .....                             | 14       |
| 3.5      | Delete Task .....                                 | 14       |
| 3.6      | Create a new Task .....                           | 14       |
| 3.7      | Generate Exam 1 .....                             | 15       |
| 3.8      | Generate Exam 2 .....                             | 16       |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 3.9      | <i>Aktionen bei Programmstart und Programmende</i> ..... | 17        |
| 3.9.1    | Programmstart .....                                      | 17        |
| 3.9.2    | Programmende .....                                       | 17        |
| <b>4</b> | <b>Spezifikation der Use-Cases</b> .....                 | <b>17</b> |
| 4.1      | <i>Aktoren</i> .....                                     | 17        |
| 4.2      | <i>Übersicht</i> .....                                   | 18        |
| 4.3      | <i>Use-Cases</i> .....                                   | 18        |
| <b>5</b> | <b>Sonstiges</b> .....                                   | <b>24</b> |
| 5.1      | <i>Anforderungen an externe Schnittstellen</i> .....     | 24        |
| 5.2      | <i>Zukünftige Erweiterungen des Systems</i> .....        | 24        |
| <b>6</b> | <b>Begriffslexikon</b> .....                             | <b>25</b> |

## 2 Beschreibung

### 2.1 Zweck des Dokuments

Diese Spezifikation beschreibt die Software **KlausurCraft**, ein Werkzeug zur Erstellung, Verwaltung und automatischen Generierung von schriftlichen Prüfungen („Klausuren“). Sie dient als Grundlage für die Implementierung, Validierung und spätere Erweiterung des Systems.

Das Dokument richtet sich an:

Entwickler und Softwarearchitekten, die für die technische Umsetzung verantwortlich sind,

Tester, die anhand dieser Spezifikation Verifikationsszenarien ableiten,

Lehrende, die die Funktionsweise der Software nachvollziehen möchten.

Ziel ist es, die Anforderungen an Funktion, Bedienung, Systemumgebung und Benutzerschnittstellen eindeutig und überprüfbar zu beschreiben.

### 2.2 Überblick über das System

**KlausurCraft** ist eine plattformunabhängige JavaFX-Applikation zur Verwaltung eines Aufgabenpools, aus dem individuelle Klausuren generiert werden können.

Das System ermöglicht es, Aufgaben nach Themengebieten zu strukturieren, Teilaufgaben und Varianten zu verwalten, sowie daraus Prüfungsdokumente im PDF-Format zu erstellen.

Hauptmerkmale des Systems:

- Import und Export von Aufgaben im XML-Format
- Verwaltung beliebig vieler Aufgaben (Tasks) und Teilaufgaben (Subtasks)
- Zuweisung von Punkten, Schwierigkeitsgrad und Sichtbarkeitsstufen
- Erstellung von Aufgabenvarianten mit unterschiedlichen Lösungen
- Automatische Klausurgenerierung mit Titel, Datum und Umfang
- Optionaler Export mit oder ohne Musterlösung
- Dunkel- und Hellmodus für komfortables Arbeiten

Das System arbeitet vollständig lokal; eine Netzwerkanbindung ist nicht erforderlich. Alle Änderungen werden unmittelbar gespeichert, sodass kein explizites „Speichern“-Kommando nötig ist.

### 2.3 Systemumgebung

#### 2.3.1 Zielplattform

KlausurCraft ist als **JavaFX-basierte Desktop-Applikation** konzipiert und auf folgenden Systemen lauffähig:

- Microsoft Windows 10 oder neuer
- macOS Monterey oder neuer
- Linux-Distributionen mit installiertem OpenJDK 17+

### 2.3.2 Hardwareanforderungen

- Prozessor: mind. 2 Kerne, 2 GHz
- Arbeitsspeicher: mind. 4 GB
- Festplattenspeicher: < 100 MB für Anwendung, zusätzlich Speicher für XML- und PDF-Dateien
- Bildschirmauflösung: mindestens 1280 × 720 Pixel (empfohlen: 1920 × 1080)

### 2.3.3 Softwarevoraussetzungen

- **Java 17 oder höher** (OpenJDK oder Oracle JDK)
- **JavaFX-Bibliothek** (im Lieferumfang enthalten)
- **OpenPDF-Bibliothek** zur PDF-Erzeugung

### 2.3.4 Installation

Die Software wird als plattformunabhängiges JAR-Paket ausgeliefert.

Installation erfolgt durch Entpacken des bereitgestellten Archivs; anschließend kann die Anwendung durch Doppelklick oder über die Konsole gestartet werden.

## 2.4 Nicht-funktionale Anforderungen

### 2.4.1 Leistungsanforderungen

- Die Anwendung wird von **einem Benutzer gleichzeitig** betrieben.
- Die Anzahl der gespeicherten Aufgaben ist nur durch den verfügbaren Systemspeicher begrenzt.
- Die Benutzeroberfläche reagiert innerhalb von 0,5 Sekunden auf Benutzereingaben.
- Das Laden einer typischen XML-Datei (< 1 MB, ~500 Tasks, ~50 Subtasks) darf maximal 2 Sekunden dauern.
- Die PDF-Erzeugung erfolgt in weniger als 5 Sekunden.

### 2.4.2 Entwurfseinschränkungen

- Programmiersprache: Java 17 (Standard Edition)
- GUI: JavaFX 25
- Datenhaltung: lokales XML-Format mit eigener Schema-Definition
- PDF-Export: OpenPDF
- Dateiformate: .xml, .pdf

### 2.4.3 Verfügbarkeit

- Das Produkt ist lokal immer verfügbar und benötigt keine Internetverbindung.
- Fehlerbehandlung erfolgt ausschließlich durch lokale Dialogmeldungen.

### 2.4.4 Sicherheit

- Alle Daten werden ausschließlich lokal gespeichert; es erfolgt **kein externer Datentransfer**.
- Benutzerrechte werden über das Betriebssystem geregelt.



- XML-Importe werden auf Strukturvalidität geprüft, um Manipulationen zu verhindern.

#### **2.4.5 Robustheit**

- Bei ungültigen Benutzereingaben (z. B. leere Felder oder negative Punktzahlen) werden informative Fehlermeldungen angezeigt.
- Die Anwendung verhindert programmseitig Aktionen, die zu Inkonsistenzen führen könnten (z. B. Löschen eines Tasks mit ungespeicherten Änderungen).
- Fehlerhafte oder unvollständige XML-Dateien werden abgefangen und mit Hinweisdialog gemeldet.

#### **2.4.6 Wartbarkeit**

- Alle Module folgen einem klaren MVC-Konzept (Model – View – Controller).
- Der Quellcode wird nach den Java-Coding-Conventions strukturiert.
- Öffentliche Klassen und Methoden sind mit JavaDoc dokumentiert.
- Das System ist erweiterbar (z. B. spätere Anbindung einer Datenbank oder Cloud-Synchronisation).

#### **2.4.7 Portabilität**

- Vollständig plattformunabhängig durch Java Virtual Machine.
- Keine nativen Bibliotheken notwendig.
- Schriftarten und Icons sind plattformneutral eingebettet.

### **2.5 Beschreibung der Benutzer**

Die Zielgruppe umfasst Lehrkräfte, Dozenten, Tutoren und wissenschaftliche Mitarbeiter, die Prüfungen erstellen oder verwalten.

Vorkenntnisse in Programmierung sind nicht erforderlich; grundlegende Computerkenntnisse werden vorausgesetzt.

Typische Benutzer:

- Lehrende, die Aufgabenpools pflegen und Klausuren zusammenstellen
- Tutoren, die Teilaufgaben vorbereiten und Lösungen prüfen
- Prüfungsbeauftragte, die fertige Klausuren exportieren

Die Benutzeroberfläche ist intuitiv aufgebaut und nutzt vertraute UI-Muster (Buttons, Tabs, Popups, Kontextdialoge).

## **2.6 Standardwerte**

### **2.6.1 Allgemeine Standardwerte**

- **Theme:** Light-Mode
- **Datei:** Wird keine XML-Datei geladen, startet die App mit einer leeren Beispielstruktur.

- **Sprache:** Englisch (Inhalte der Klausur nach Nutzereingabe)
- **Speicherort:** Letzter verwendeter XML-Pfad wird gespeichert.

### **2.6.2 Standardwerte für Tasks**

- Name: *New Task*
- Punktzahl: 10
- Schwierigkeitsgrad: „medium“
- Sichtbarkeit: „exam“

### **2.6.3 Standardwerte für Subtasks**

- Name: *New Subtask*
- Punktzahl: 5
- Schwierigkeitsgrad: „easy“
- Sichtbarkeit: „exam“
- Enthält zunächst keine Varianten

### **2.6.4 Standardwerte für Varianten**

- Variante enthält ein leeres Textfeld für Aufgabenstellung und Lösung
- Kein Musterlösungshaken gesetzt

### **2.6.5 Standardwerte für generierte Exams**

- Titel: „Sample Exam“
- Datum: heutiges Systemdatum
- Enthaltene Aufgaben: keine (Benutzer wählt manuell aus)
- Musterlösung: ausgeschaltet

# 3 Spezifikation der Benutzeroberfläche

## 3.1 Übersicht

Hier wird eine Übersicht aller Fenster und Popups der Applikation KlausurCraft dargestellt.

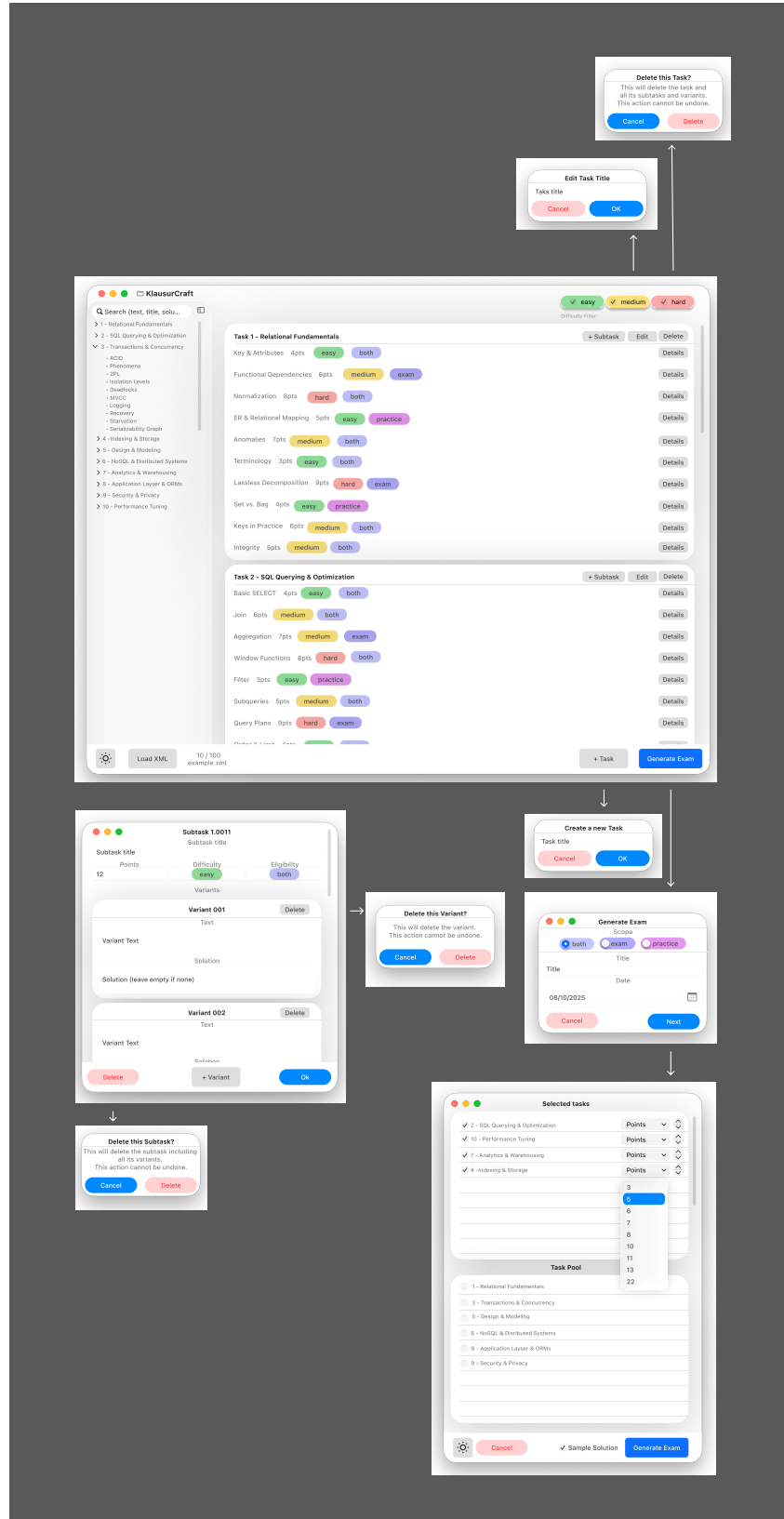


Abbildung 1: Übersicht

## 3.2 Hauptoberfläche

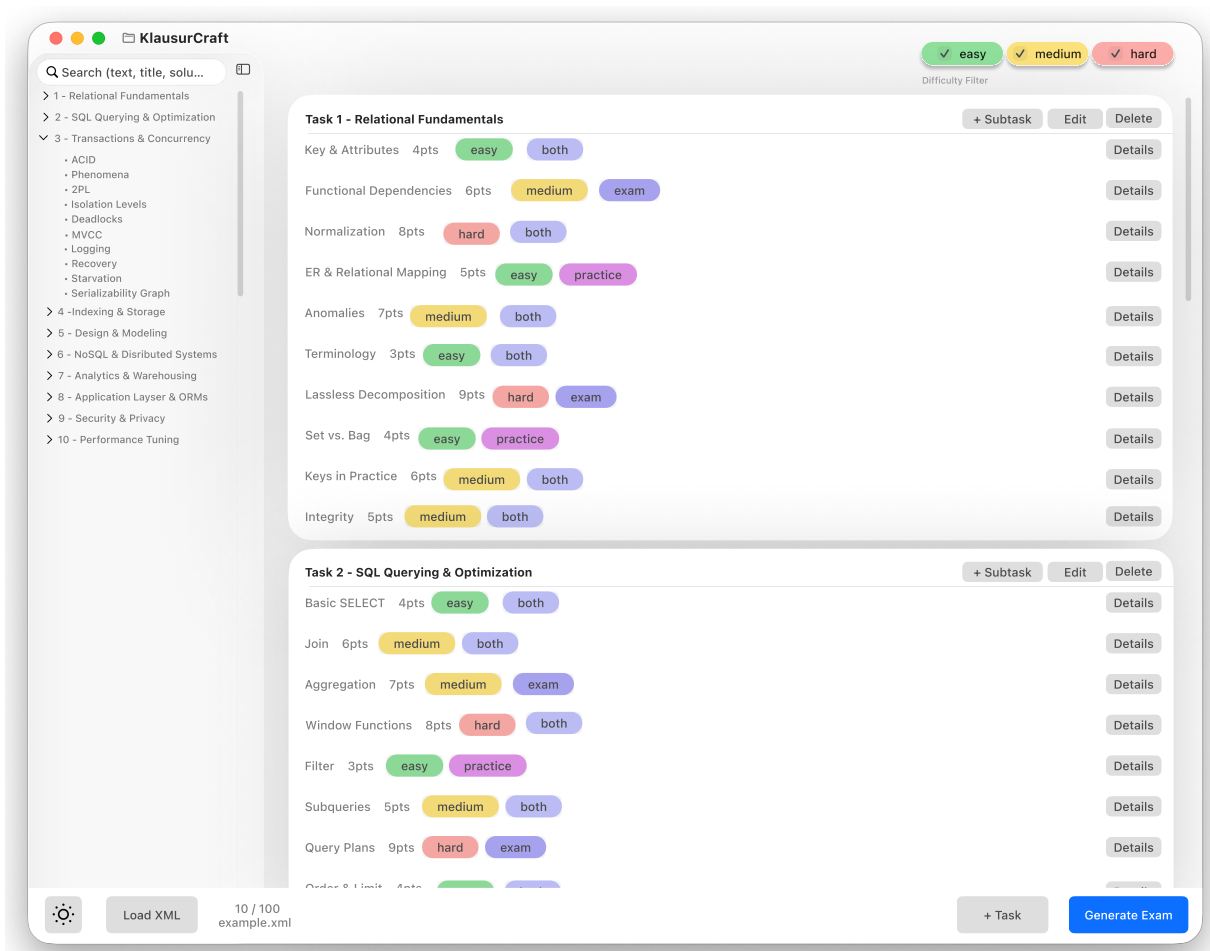


Abbildung 2:

Das Hauptfenster von *KlausurCraft* bildet die zentrale Arbeitsoberfläche.

Es unterteilt sich – analog zu modernen Entwicklungs- und Verwaltungswerkzeugen – in drei Hauptbereiche:

1. **Navigationsleiste (links)**

Enthält Suchfeld, Filter und die hierarchische Liste aller geladenen Tasks.

2. **Hauptbereich (zentral)**

Zeigt die Aufgaben- und Subtask-Struktur, Detailansichten und Bearbeitungsdialoge.

3. **Fußleiste (unten)**

Stellt allgemeine Steuerungselemente, Statusinformationen und den Dark-/Light-Mode bereit.

Beim Start ist der Light-Mode aktiv, die Navigation ausgeklappt und ein leerer Beispiel-Task sichtbar.

### 3.2.1 Navigationsleiste

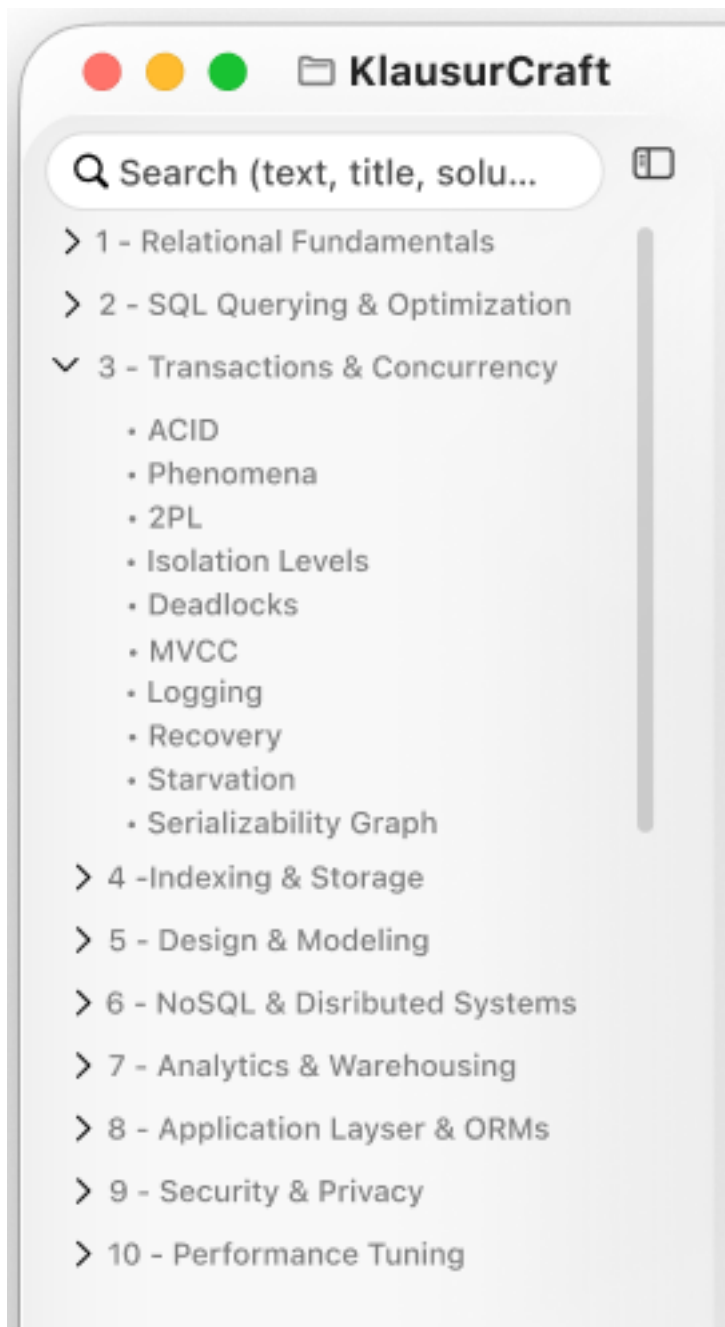


Abbildung 3: Navigationsleiste

Die Navigationsleiste enthält:

- ein **Suchfeld** („Search...“) zur Volltextsuche in Task-Titeln, Subtask-Texten oder Solutions,
- eine hierarchische **Baumdarstellung aller Tasks** mit deren **Subtasks** zum ausklappen.

Ein Klick auf einen Task zeigt im Hauptbereich seine Subtasks, ein weiterer Klick öffnet den Detail-View.

Die Leiste lässt sich über ein Chevron-Symbol links einklappen.

Die Such- und Filterfelder bleiben auch im eingeklappten Zustand funktionsfähig.

### 3.2.2 Hauptbereich

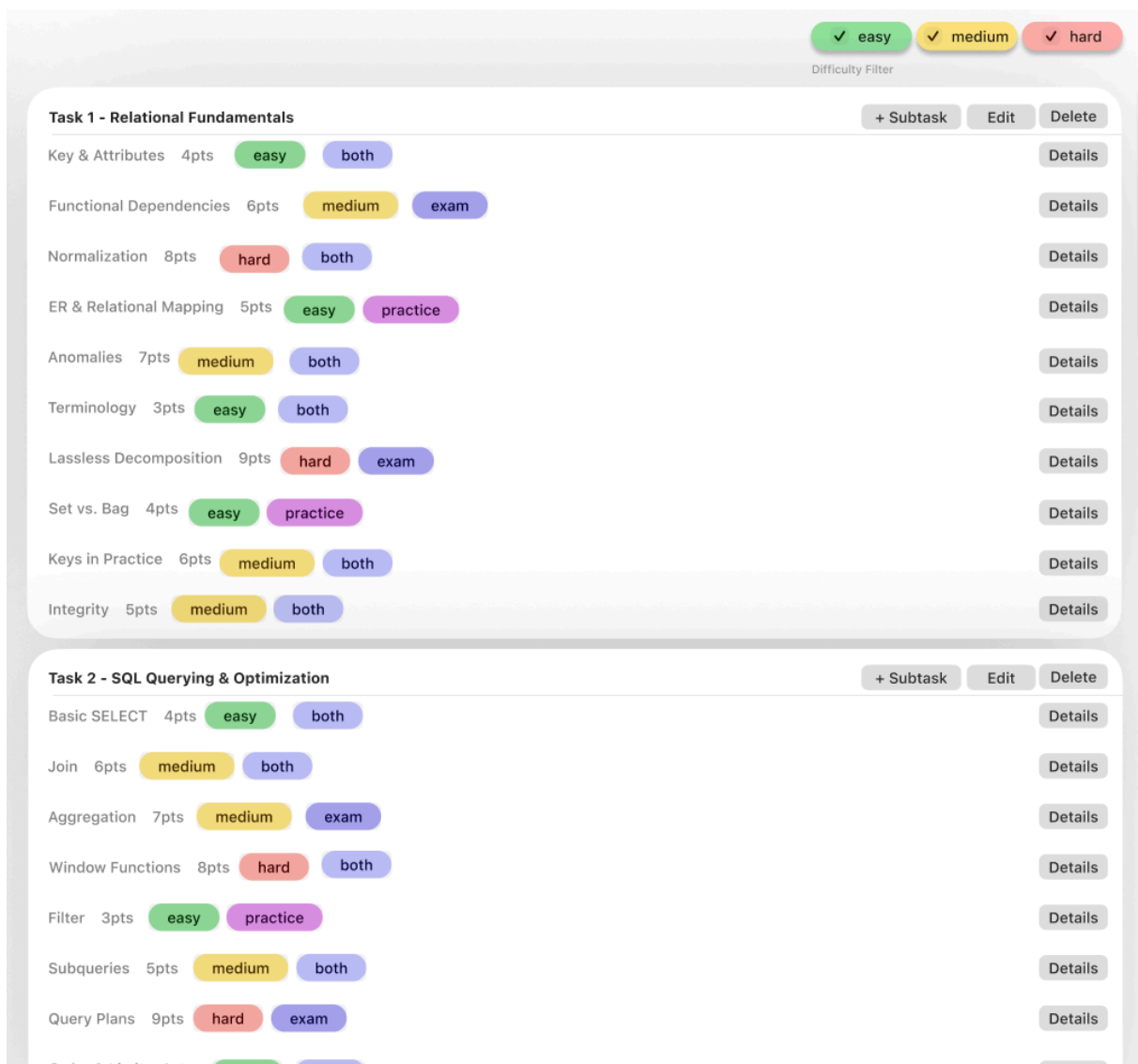


Abbildung 4: Hauptbereich

Der Hauptbereich nimmt etwa zwei Drittel der Fensterbreite ein.

Er zeigt die aktuell gefilterten Tasks mitsamt Subtasks, Punkten, Schwierigkeits- und Sichtbarkeitsgrad.

Jeder Task ist in einem Card-Layout dargestellt, bestehend aus:

- Task-Titel (editable per „Edit“-Button)
- Buttons „+Subtask“, „Edit“, „Delete“
- Liste aller zugehörigen Subtasks

Jeder **Subtask-Eintrag** zeigt:

- Titel, Punktzahl, Schwierigkeitsstufe, Sichtbarkeit
- Buttons „Detail“ zum anzeigen und bearbeiten der Varianten mit Text und Lösungen

Scrollleisten ermöglichen das Navigieren bei vielen Aufgaben.

### 3.2.3 Fußleiste



Abbildung 5: Fußleiste

Am Fuß der Applikation befindet sich eine horizontale Toolbar mit folgenden Buttons (von links nach rechts):

1. **Switch Dark/Light Mode** – er ermöglicht das Wechseln des Anzeigethemas
2. **Load XML** – öffnet eine bestehende Aufgaben-Datei.
3. **+Task** – öffnet das „new Task“ Popup
4. **Generate Exam** – öffnet den Klausurgenerator.

Zudem sieht man den Name des aktuell geladen XML-Files und wie viele Tasks und Subtasks geladen werden konnten.

### 3.3 Subtask

Über die „Detail“ oder „+Subtask“ Button im Hauptfenster kann das folgende Fenster aufgerufen werden:

Abbildung 6: + Subtask and Detail Button

In diesem Fenster kann eine neue **Subtask** angelegt oder eine bestehende bearbeitet werden.

Es lassen sich **Titel**, **Punktzahl**, **Schwierigkeitsgrad** und **Sichtbarkeitsstufe** (Eligibility) festlegen.

Darunter befindet sich der Bereich **Variants**, in dem verschiedene Varianten einer Aufgabe hinzugefügt werden können.

Jede Variante enthält ein **Textfeld** für den Aufgabentext sowie ein optionales **Lösungsfeld** (Solution).

Über den Button „+ **Variant**“ lassen sich neue Varianten ergänzen.

Mit „**Ok**“ wird das Fenster geschlossen. Das Speichern passiert automatisch im Hintergrund.



Der „**Delete**“-Button entfernt die aktuell ausgewählte Subtask.

### 3.3.1 Delete Subtask

Der „**Delete**“-Button in der Fußleiste des **Subtask**- oder **Detail View** öffnet das folgende Bestätigungsfenster:

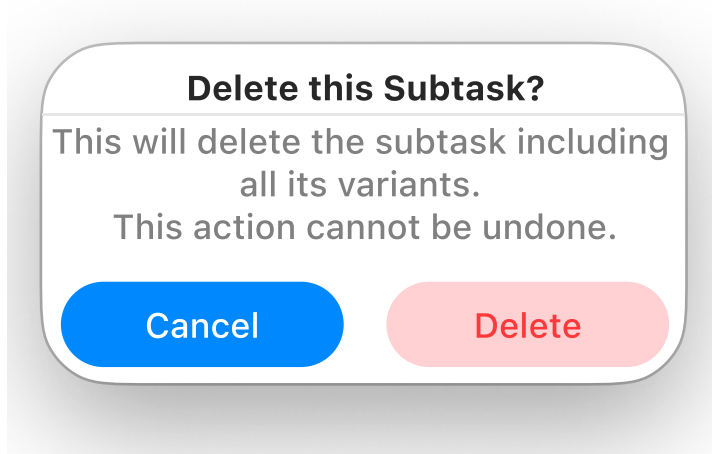


Abbildung 7: Delete Subtask

In diesem Popup kann das Löschen der gesamten **Subtask** einschließlich aller zugehörigen Varianten bestätigt oder abgebrochen werden. Nach der Bestätigung wird die Subtask dauerhaft entfernt – diese Aktion kann nicht rückgängig gemacht werden.

### 3.3.2 Delete Variant

Über den „**Delete**“-Button im **Subtask**- oder **Detail View** innerhalb einer Variante erscheint folgendes Bestätigungsfenster:

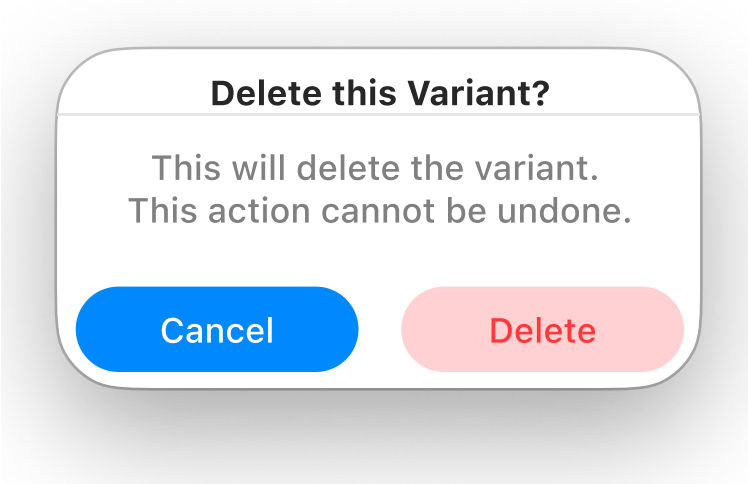


Abbildung 8: Delete Variant

Hier kann das Löschen der ausgewählten **Variante** bestätigt oder abgebrochen werden. Auch dieser Vorgang ist **nicht rückgängig** zu machen.

### 3.4 Edit Task Title

Über den „**Edit**“-Button im Hauptfenster, direkt neben dem **Tasktitel**, kann der Titel einer bestehenden **Task** über das folgende Popup-Fenster geändert werden:

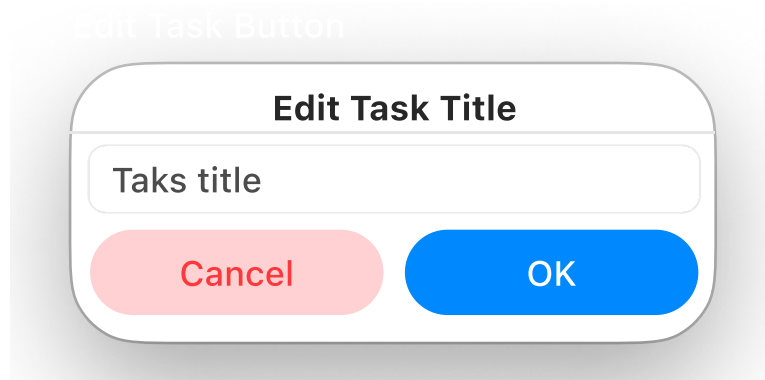


Abbildung 9: Edit Task Title

Im angezeigten Dialog kann ein neuer Titel eingegeben und mit „**OK**“ bestätigt werden. Durch „**Cancel**“ wird der Vorgang ohne Änderung abgebrochen.

### 3.5 Delete Task

Der „**Delete**“-Button im Hauptfenster, ebenfalls neben dem **Tasktitel**, ermöglicht das Löschen einer gesamten **Task** einschließlich aller zugehörigen **Subtasks** und **Varianten**. Beim Klicken erscheint folgendes Bestätigungsfenster:

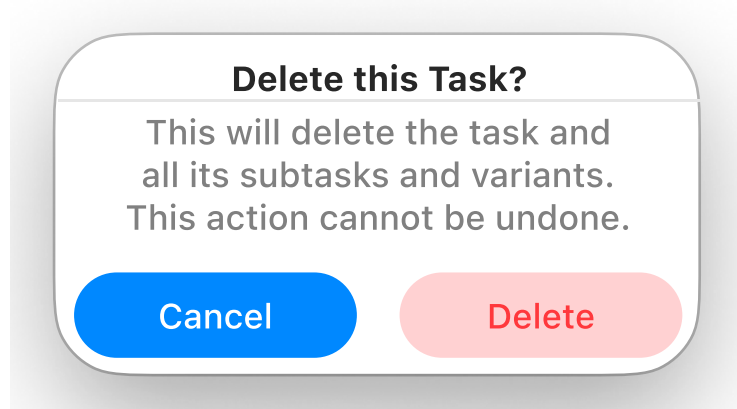


Abbildung 10: Delete Task

Hier kann das Löschen der ausgewählten Task bestätigt oder abgebrochen werden. Nach der Bestätigung wird die Task dauerhaft entfernt – dieser Vorgang kann **nicht rückgängig gemacht** werden.

### 3.6 Crete a new Task

Über den „**+ Task**“-Button in der Fußleiste kann folgender Dialog zum Erstellen einer neuen **Task** geöffnet werden:

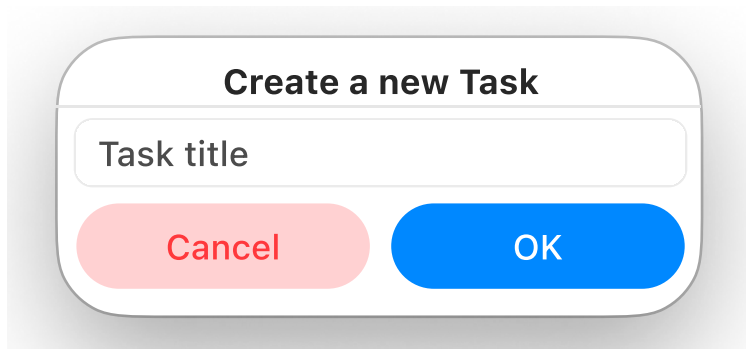


Abbildung 11: Create a new Task

In diesem Fenster kann der Benutzer einen individuellen **Tasktitel** festlegen. Der Vorgang wird mit „**OK**“ bestätigt oder mit „**Cancel**“ abgebrochen. Nach der Bestätigung erscheint die neue Task im Hauptfenster.

### 3.7 Generate Exam 1

Über den „**Generate Exam**“-Button in der Fußleiste des **Hauptmenüs** kann der Hauptprozess zur Erstellung einer Klausur gestartet werden. Es öffnet sich das folgende Fenster:

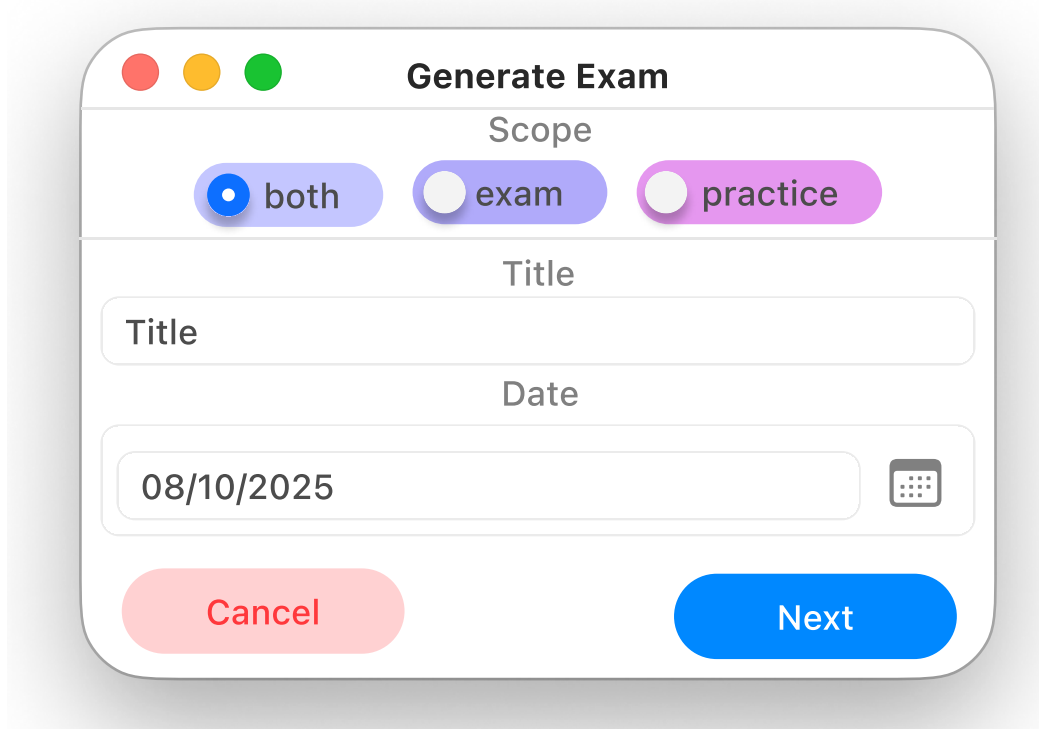


Abbildung 12: Generate Exam 1

Hier kann der **Scope** (z. B. *exam*, *practice* oder *both*), der **Titel** sowie das **Datum** des Exams festgelegt werden.

Mit „**Next**“ wird der Vorgang fortgesetzt, während „**Cancel**“ den Prozess abbricht.

### 3.8 Generate Exam 2

The interface is titled "Generate Exam 2" and is divided into two main sections: "Selected tasks" and "Task Pool".

**Selected tasks:** This section contains a list of tasks that have been selected for the exam. Each task is preceded by a checkmark. The tasks are:

- ✓ 2 - SQL Querying & Optimization
- ✓ 10 - Performance Tuning
- ✓ 7 - Analytics & Warehousing
- ✓ 4 - Indexing & Storage

Each task has a "Points" dropdown menu next to it. A dropdown menu is currently open, showing a list of points: 3, 5, 6, 7, 8, 10, 11, 13, and 22. The value "5" is highlighted in blue.

**Task Pool:** This section contains a list of tasks that are available for selection. Each task is preceded by an unchecked checkbox. The tasks are:

- 1 - Relational Fundamentals
- 3 - Transactions & Concurrency
- 5 - Design & Modeling
- 6 - NoSQL & Distributed Systems
- 8 - Application Layer & ORMs
- 9 - Security & Privacy

At the bottom of the interface, there is a row of controls:

- A gear icon for settings.
- A red "Cancel" button.
- A checkbox labeled "Sample Solution" which is currently checked.
- A blue "Generate Exam" button.

Abbildung 13: Generate Exam 2

In diesem Fenster werden alle im **Task Pool** verfügbaren Aufgaben angezeigt.

Über die **Checkbox** neben jeder Aufgabe kann entschieden werden, ob sie in das aktuelle **Exam** aufgenommen wird.

Bereits ausgewählte Aufgaben können durch Deaktivieren der Checkbox wieder aus dem Exam entfernt und in den Task Pool zurückgegeben werden.

Für jede ausgewählte Aufgabe muss anschließend im **Dropdown-Menü** eine **Punktzahl** festgelegt werden.

Mit den Pfeilen auf der rechten Seite lässt sich zudem die **Reihenfolge** der Aufgaben innerhalb des Exams anpassen.

In der Fußleiste kann optional die Erstellung einer **Sample Solution** aktiviert werden. Mit dem Button „**Generate Exam**“ wird die Klausur erstellt, während „**Cancel**“ den Vorgang abbricht.

## **3.9 Aktionen bei Programmstart und Programmende**

### **3.9.1 Programmstart**

Beim Programmstart wird das Hauptfenster angezeigt. Wenn die Applikation schonmal verwendet wurde, wird automatisch das zuletzt geöffnete XML-File geladen wenn es sich noch an der selben Stelle befindet.

### **3.9.2 Programmende**

Beim schließen des Programms werden alle Fenster geschlossen. Dadurch das die Speicherung in das XML-File immer direkt nach Nutzereingabe erfolgt muss sonst keine Aktion durchgeführt werden.

## **4 Spezifikation der Use-Cases**

### **4.1 Aktoren**

Der einzige Akteur ist ein **Dozent**.

Ein Dozent ist eine lehrende oder prüfungsverantwortliche Person, die Aufgaben, Teilaufgaben und Klausuren erstellt.

Optional kann ein Dozent auch Tutor oder wissenschaftlicher Mitarbeiter sein.

Er verfügt über solide Computerkenntnisse, benötigt aber keine Programmiererfahrung. Alle Aktionen erfolgen über die grafische Benutzeroberfläche von *KlausurCraft*.

## 4.2 Übersicht

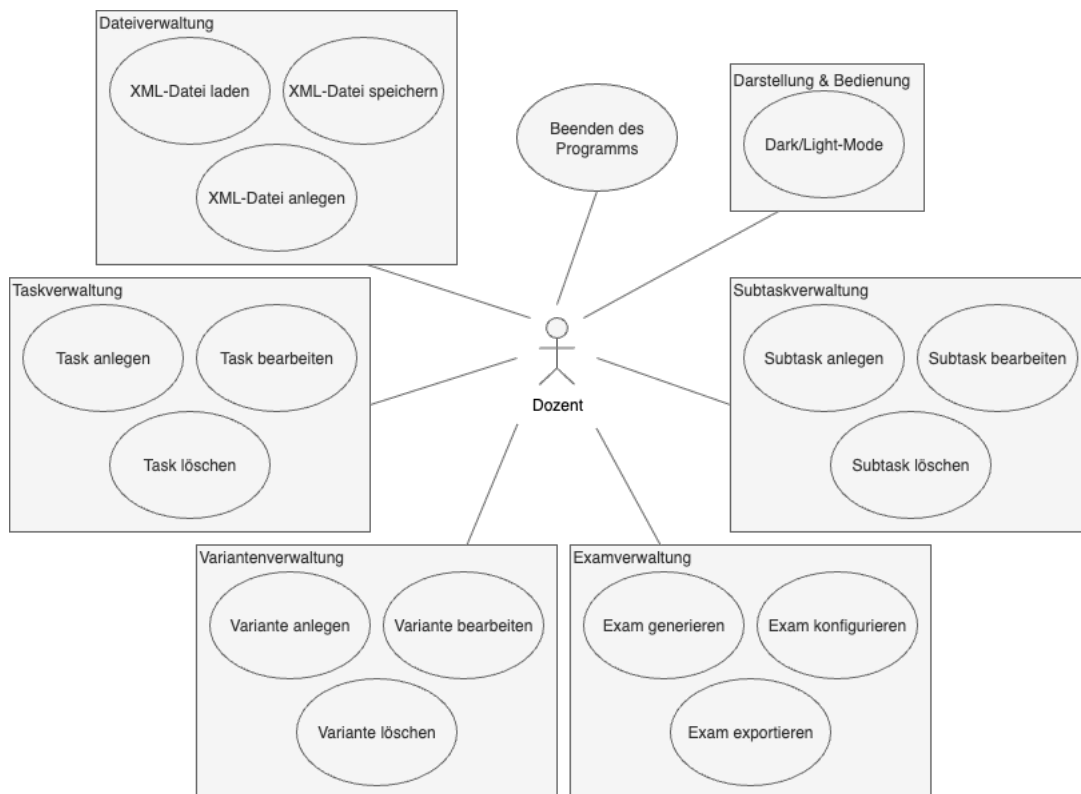


Abbildung 14: Use-Cases

| Gruppe                  | Inhalt                                         |
|-------------------------|------------------------------------------------|
| Dateiverwaltung         | XML-Dateien anlegen, bearbeiten, löschen       |
| Taskverwaltung          | Tasks anlegen, bearbeiten, löschen             |
| Subtaskverwaltung       | Subtasks anlegen, bearbeiten, löschen          |
| Variantenverwaltung     | Variante anlegen, bearbeiten, löschen          |
| Exam-Erstellung         | Klausur generieren, konfigurieren, exportieren |
| Darstellung & Bedienung | Dark/Light-Mode                                |
| Beenden des Programms   | Sicheres Schließen ohne Datenverlust           |

Jeder Use-Case enthält **Vorbedingungen**, den **regulären Ablauf**, **Nachbedingungen** und ggf. **Alternative Abläufe**.

## 4.3 Use-Cases

### 3.3.1 Neue XML-Datei anlegen

#### **Vorbedingung:**

Die Anwendung ist gestartet und zeigt das Hauptfenster.

#### **Regulärer Ablauf:**

1. Der Dozent klickt auf „New XML“.
2. Es erscheint ein Dialog zur Benennung der neuen Datei.

3. Nach Bestätigung wird eine leere Aufgabenstruktur mit einem Beispiel-Task angelegt.
4. Standardwerte werden gesetzt.
5. Die neue Datei wird automatisch gespeichert und geöffnet.

**Nachbedingung:**

Ein neues XML-Dokument mit einer Beispielstruktur ist aktiv.

**Alternativer Ablauf:**

Wenn der Benutzer den Dialog abbricht, bleibt das bisherige Projekt geöffnet. Der Nutzer kann auch händisch eine neue XML-Datei anlegen.

**3.3.2 XML-Datei laden****Vorbedingung:**

Das Hauptfenster ist sichtbar.

**Regulärer Ablauf:**

1. Klick auf „Load XML“.
2. Dateiauswahldialog öffnet sich.
3. Benutzer wählt eine XML-Datei.
4. *KlausurCraft* prüft Struktur und Schema.
5. Tasks und Subtasks werden geladen und in der Navigationsleiste angezeigt.
6. Erfolgreiches Laden wird im Footer bestätigt.

**Nachbedingung:**

Die ausgewählte XML-Datei ist vollständig geladen und aktiv.

**Alternativer Ablauf:**

- Ungültige Dateien werden mit Meldung „*File could not be loaded*“ abgewiesen.
- Wird keine Datei gewählt, bleibt die bisherige geöffnet.

**3.3.3 XML-Datei speichern****Vorbedingung:**

Mindestens eine Aufgabe ist geladen oder verändert.

**Regulärer Ablauf:**

1. Die Datei wird bei jeder Nutzereingabe und Änderung automatisch in die geladene XML-Datei gespeichert.

**Nachbedingung:**

Alle Änderungen sind persistiert.

**Alternativer Ablauf:**

Falls keine Schreibrechte bestehen, erscheint der Dialog „*File cannot be saved*“; Änderungen bleiben im Speicher.

**3.3.4 Task erstellen**

**Vorbedingung:**

Ein XML-Projekt ist geöffnet.

**Regulärer Ablauf:**

1. Klick auf „+Task“.
2. Dialog öffnet sich.
3. Eingabe des Task-Titels, Bestätigung mit „OK“.
4. Anzeige aktualisiert sich automatisch.

**Nachbedingung:**

Ein neuer Task ist in der Navigationsleiste sichtbar.

**Alternativer Ablauf:**

Abbruch des Dialogs erzeugt keine neue Aufgabe.

**3.3.5 Task bearbeiten****Vorbedingung:**

Mindestens eine Task existiert.

**Regulärer Ablauf:**

1. Klick auf „Edit“ neben dem Task-Titel.
2. Popup erscheint mit Eingabefeld.
3. Benutzer gibt neuen Titel ein.
4. Bestätigung mit „OK“ → Titel im XML aktualisiert.
5. Oberfläche aktualisiert sich unmittelbar.

**Nachbedingung:**

Task-Titel ist geändert und gespeichert.

**Alternativer Ablauf:**

Bei leerer Eingabe bleibt alter Titel bestehen.

**3.3.6 Task löschen****Vorbedingung:**

Mindestens eine Task existiert.

**Regulärer Ablauf:**

1. Klick auf „Delete“ neben der Task.
2. Popup erscheint.
3. Benutzer bestätigt.
4. Task und alle zugehörigen Subtasks und Varianten werden entfernt.

**Nachbedingung:**

Task ist gelöscht, Anzeige aktualisiert.



**Alternativer Ablauf:**

Bei „Cancel“ wird kein Element gelöscht.

**3.3.7 Subtask anlegen****Vorbedingung:**

Ein Task ist aktiv ausgewählt.

**Regulärer Ablauf:**

1. Klick auf „+Subtask“.
2. Subtask-Editor öffnet sich.
3. Eingabe von Titel, Punkten, Schwierigkeitsgrad.
4. Eingabe von N-Varianten mit Aufgabentext und optionaler Lösung
5. Klick auf „Save“.
6. Subtask erscheint unterhalb der Task.

**Nachbedingung:**

Neue Subtask ist im Task-Baum sichtbar und gespeichert.

**Alternativer Ablauf:**

Bei fehlerhaften Eingaben (leerer Titel, negative Punkte) erscheint ein Hinweisdialog, Speichern bleibt gesperrt.

**3.3.8 Subtask bearbeiten****Vorbedingung:**

Eine Subtask existiert.

**Regulärer Ablauf:**

1. Klick auf „Detail“.
2. Subtask-Editor öffnet sich.
3. Benutzer passt Parameter an.
4. Klick auf „Save“.
5. XML-Datei wird aktualisiert.

**Nachbedingung:**

Änderungen an der Subtask sind übernommen.

**Alternativer Ablauf:**

Bei ungültigen Eingaben wird letzter gültiger Wert wiederhergestellt.

**3.3.9 Subtask löschen****Vorbedingung:**

Eine Subtask ist geöffnet.

**Regulärer Ablauf:**

1. Klick auf „Delete“.

2. Bestätigungsdialog.
3. Klick auf „Delete“.
4. Subtask wird entfernt.

**Nachbedingung:**

Subtask und alle Varianten sind gelöscht.

**Alternativer Ablauf:**

Bei „Cancel“ keine Aktion.

### **3.3.10 Variante anlegen**

**Vorbedingung:**

Eine Subtask ist aktiv.

**Regulärer Ablauf:**

1. Klick auf „+Variant“.
2. Variant-Editor öffnet sich.
3. Eingabe von Aufgaben- und Lösungstext.
4. Klick auf „Save“.

**Nachbedingung:**

Neue Variante wird im Subtask-Editor angezeigt.

**Alternativer Ablauf:**

Abbruch → keine Änderung.

### **3.3.11 Variante bearbeiten**

**Vorbedingung:**

Die zu bearbeitende Variante ist bereits angelegt.

**Regulärer Ablauf:**

1. Klick auf „Details“ der Subtask im Hauptfenster.
2. Detaildialog öffnet sich.
3. Variante sowie Lösung einfach bearbeiten.

**Nachbedingung:**

Variante wird in XML-Datei geändert.

**Alternativer Ablauf:**

Bei „Cancel“ keine Aktion.

### **3.3.11 Variante löschen**

**Vorbedingung:**

Eine Variante ist ausgewählt und sie ist nicht die einzige Variante der Subtask.

**Regulärer Ablauf:**

4. Klick auf „Delete Variant“.
5. Bestätigungsdialog öffnet sich.
6. Benutzer bestätigt.

**Nachbedingung:**

Variante wird aus XML gelöscht.

**Alternativer Ablauf:**

Bei „Cancel“ keine Aktion.

**3.3.12 Klausur generieren****Vorbedingung:**

Mindestens eine Task vorhanden.

**Regulärer Ablauf:**

1. Klick auf „Generate Exam“.
2. Dialog 1 (Titel, Datum, Option Musterlösung).
3. Klick auf „Next“.
4. Dialog 2 zeigt Task-Liste mit Checkboxes.
5. Benutzer wählt Aufgaben aus, legt Punktzahlen und Reihenfolge fest.
6. Klick auf „Generate“.
7. System erzeugt PDF-Datei.

**Nachbedingung:**

PDF wurde erstellt und im Ausgabeordner gespeichert.

**Alternativer Ablauf:**

Keine Auswahl → Meldung „*Please select at least one task.*“

Abbruch → Dialog schließt sich ohne Änderung.

**3.3.13 Dark/Light-Mode umschalten****Vorbedingung:**

Programm läuft.

**Regulärer Ablauf:**

1. Klick auf Theme-Button im Footer.
2. Theme wechselt sofort (CSS-Umschaltung).
3. Einstellung wird in Konfigurationsdatei gespeichert.

**Nachbedingung:**

Neues Erscheinungsbild bleibt bis zum nächsten Start erhalten.

**Alternativer Ablauf:**

Keine.

### 3.3.14 Programm beenden

**Vorbedingung:**

Hauptfenster geöffnet.

**Regulärer Ablauf:**

1. Benutzer klickt auf das Schließen-Symbol.
2. Anwendung prüft, ob XML-Datei existiert.
3. Da alle Änderungen direkt gespeichert sind, erfolgt keine Nachfrage.
4. Programm beendet sich ordnungsgemäß.

**Nachbedingung:**

Fenster geschlossen, keine offenen Prozesse.

**Alternativer Ablauf:**

Systemfehler → Fehlermeldung „Unexpected shutdown“ wird geloggt.

## 5 Sonstiges

### 5.1 Anforderungen an externe Schnittstellen

KlausurCraft ist eine **lokal ausgeführte Java-Applikation**, die keine externe Netzwerkanbindung benötigt.

### 5.2 Zukünftige Erweiterungen des Systems

**Undo/Redo-Funktion:**

Ermöglicht das Rückgängigmachen und Wiederherstellen von Bearbeitungsschritten.

**Vorlagen und Designs:**

Unterstützung für unterschiedliche Layout- und Formatvorlagen beim PDF-Export.

**Aufgabenstatistik:**

Automatische Auswertung von Schwierigkeitsgraden, Punkten und Häufigkeiten.

**Import/Export-Optionen:**

Unterstützung zusätzlicher Formate (CSV, JSON, Markdown).

**Mehrsprachigkeit:**

Möglichkeit, die Benutzeroberfläche zwischen Deutsch und Englisch umzuschalten.

**Datenbankanbindung:**

Umstieg von XML auf SQLite oder PostgreSQL für höhere Skalierbarkeit.

**REST-API:**

Webschnittstelle zur Integration in Lernplattformen (z. B. Moodle, Ilias).

**Erweiterte PDF-Generierung:**

Layoutverbesserungen, Seitenzahlen, Kopf- und Fußzeilen, Institutionen-Branding.

**Plugin-System:**

Erweiterbarkeit um neue Aufgabentypen oder spezielle Generator-Module.

## 6 Begriffslexikon

### A

**Action Button**

Ein Bedienelement (z. B. *Save*, *Delete*), das beim Anklicken eine Aktion auslöst.

**Application Window**

Das Hauptfenster der Applikation, in dem alle Aufgaben und Menüs dargestellt werden.

**Auto-Save**

Automatische Speicherung aller Änderungen in der geöffneten XML-Datei ohne manuelle Bestätigung.

---

### C

**Cancel Dialog**

Ein Popup, das eine laufende Aktion abbricht und ohne Änderungen zum vorherigen Fenster zurückkehrt.

**Card Layout**

Darstellungsform der Tasks im Hauptbereich, bei der jede Aufgabe als einzelne „Karte“ angezeigt wird.

---

### D

**Dark Mode**

Darstellung der grafischen Oberfläche mit dunklen Farben zur besseren Lesbarkeit bei geringem Umgebungslicht.

**Delete Confirmation**

Popup-Dialog, der das Löschen einer Entität (z. B. Task oder Variant) bestätigt.

**Detail View**

Anzeigefenster zur Bearbeitung eines Subtasks oder einer Variante.

---

### E

**Egibility**

Sichtbarkeit der einzelnen Subtasks. Entweder werden sie für eine echtes Exam, eine practice Exam oder für beides genutzt.

### **Exam Generator**

Zentrales Modul zur Erstellung einer vollständigen Klausur aus vorhandenen Aufgaben.

### **Exam Scope**

Definiert den in die Klausur aufzunehmenden Aufgabenbereich (z. B. nach der Sichtbarkeit von exam oder practice).

---

## **F**

### **Filter Options**

Checkboxen im Navigationsbereich zur Einschränkung der sichtbaren Tasks nach Kriterien wie Schwierigkeitsgrad.

### **Footer**

Unterer Bereich des Hauptfensters mit Statusmeldungen und Steuerungselementen (z. B. Theme-Umschalter).

---

## **G**

### **Generate Exam**

Hauptbefehl zur Zusammenstellung und PDF-Erzeugung einer Klausur.

### **Graphical User Interface (GUI)**

Visuelle Bedienoberfläche von *KlausurCraft*, entwickelt mit JavaFX.

---

## **H**

### **Header**

Oberer Teil eines Fensters oder Dialogs, der Titel, Symbole und gegebenenfalls Schließen-Buttons enthält.

### **Hotkey**

Tastenkombination zur Ausführung einer häufig genutzten Aktion (z. B. *Ctrl + D* für den Wechsel zwischen Dark und Light Mode).

---

## **I**

### **Icon**

Grafisches Symbol zur schnellen Identifikation einer Funktion oder Datei.

### **Import XML**

Ladevorgang, bei dem eine bestehende Aufgaben-XML-Datei eingelesen wird.

### **Input Field**

Eingabefeld für Text- oder Zahlenwerte.

---

## L

### **Light Mode**

Darstellung der Benutzeroberfläche mit hellem Hintergrund und dunkler Schrift.

### **List View**

Darstellung mehrerer Datenelemente in Listenform, z. B. Varianten einer Subtask.

---

## M

### **Main Menu**

Zentrale Ansicht der Applikation mit Navigationsleiste, Aufgabenübersicht und Fußzeile.

### **Modal Window**

Fenster, das den Fokus des Benutzers erfordert und Interaktionen mit dem Hauptfenster vorübergehend sperrt.

---

## N

### **Navigation Pane**

Linke Seitenleiste mit Such- und Filterfunktionen.

### **Node**

Einzelnes Element im Aufgabenbaum (Task, Subtask).

### **Notification**

Kurze Statusmeldung (z. B. „Exam generated successfully“).

---

## O

### **Open Dialog**

Systemfenster zum Auswählen einer bestehenden Datei.

### **OpenPDF**

Bibliothek zur Erzeugung und Bearbeitung von PDF-Dokumenten in Java.

### **Overlay**

Transparente Schicht, die bei geöffneten Dialogen das Hauptfenster abdunkelt.

---

## P

### **Popup**

Kleines Fenster zur Bestätigung, Warnung oder Eingabe.

---

## R

### **Right-Click Menu**

Kontextmenü, das bei einem Rechtsklick geöffnet wird.

---

## **S**

### **Sample Solution**

Eine Beispiel- oder Musterlösung, die optional in der Klausur angezeigt werden kann.

### **Scroll Bar**

UI-Element zur Navigation bei langen Listen oder Texten.

### **Search Field**

Eingabefeld in der Navigationsleiste zum Durchsuchen aller Aufgaben.

### **Status Message**

Kurzer Text im Header, der den Erfolg oder Fehler einer Aktion anzeigt.

### **Subtask**

Eine Teilaufgabe innerhalb einer übergeordneten Task.

---

## **T**

### **Task**

Eine Hauptaufgabe oder Themenblock innerhalb des Aufgabenpools.

### **Theme**

Gesamterscheinungsbild der Applikation, z. B. Light- oder Dark-Mode.

### **Toolbar**

Horizontale Leiste mit häufig genutzten Steuerungselementen

---

## **U**

### **User Interface**

Visuelle und interaktive Elemente, mit denen der Benutzer arbeitet.

---

## **V**

### **Variant**

Alternative Formulierung einer Subtask mit eigenem Aufgabentext und Lösung.

### **Validation Error**

Hinweis auf ungültige Eingabe (z. B. leeres Feld).

---

## **W**

### **Warning Dialog**

Popup, das auf einen möglichen Datenverlust oder eine kritische Aktion hinweist.



**Workflow**

Abfolge der logischen Schritte bei der Erstellung einer Klausur: Aufgaben auswählen, Subtasks bearbeiten, Varianten festlegen, Exam generieren.